

Ambito de decision (Objetos / Arquitectura / Persistencia / Otro)	Componente/s impactado/s	Decisión	Otras Alternativas	Justificación de la decisión			
Objetos	Contacto, Colaborador	Contacto como clase abstracta para cada medio	Ser un string dentro de la lista mediosDeContacto	Menor sobrecarga: desarrollar la logica notificar() en otras clases para no sobrecargar la clase colaborador. Cada medio tendria su forma de notificar al colaborador			
Objetos	Heladera	Poner atributo capacidad para heladera	1. Para cada vianda tener un atributo tamaño 2. Obtener la capacidad en base a la cantidad de viandas que tiene dentro	Eficiencia: Basados en el enunciado sabemos que se normalizo el tamaño de las viandas por lo que poner un Int(cantViandas) como atributo ya seria suficiente.			
Objetos	Colaboracion, DonacionViandas, DonacionDinero, DistribucionViandas, etc	todas las donaciones herencian de Colaboracion	que no exista herencia y cada una sea independiente	La herencia ayuda a organizar y estructurar el código de manera más lógica y coherente. Ademas, ayuda a reutilizar el código y evitar la duplicación			
Objetos	Tarjeta, TarjetaColaborador, TarjetaVulnerable, Heladera	Crear una clase abstracta Tarjeta con subclases especificas para diferentes tipos de tarjetas, como TarjetaColaborador para donaciones y TarjetaVulnerable para extracciones de viandas	Modelar una única clase Tarjeta que maneje todos los casos de uso mediante atributos o banderas que diferencien su comportamiento	Polimorfismo y Principio de Sustitución de Liskov: La herencia permite que cada tipo de tarjeta implemente su propio comportamiento (por ejemplo, el método de apertura en la heladera), haciendo el código más modular y reutilizable. Se evitan condicionales complicados dentro de una sola clase. Además, es más fácil añadir nuevas variantes de tarjetas en el futuro sin afectar el código existente. Esto también mejora la cohesión, ya que cada subclase tiene una única responsabilidad clara.			
Objetos	Tarjeta	Poner un atributo cantUsosHoy en donde vaya aumentando por cada uso de tarjeta que le da la persona Vulnerables. y que con un Timer programado automaticamente a las 00:00 se reinicie a 0. De esa forma al querer extraer una vianda verificara que ese atributo sea menor a la cantUsosDiariosPermitidos().	1. que la cantidad de usos diarios se vaya restando de una variable por cada uso y, en caso de llegar a 0, no permita sacar viandas. Sin embargo, de esta forma deberiamos setear la variable de la cual se van restando los usos por lo que pierde flexibilidad la misma. 2. hacer que el sistema constantemente verifica la hora para que, al momento de ser las 00:00, se reinicie la cantUsosHoy. Pero esto genera una sobrecarga innecesaria en el sistema.	Menor sobrecarga: con el Timer solo actuara a las 00:00 asique perdemos sobrecarga en el sistema. Flexibilidad: Con el method cantUsosDiariosPermitidos() ganamos flexibilidad ya que realiza el calculo de 4 + cantMenoresAcargo pudiendo esta formula cambiar en el futuro.			
Objetos	Tarjeta	modelar la clase tarjeta conociendo a su persona responsable	1. Crear la clase y asignarla como atributo a la persona vulnerable 2. No modelarla ya que seria una representacion de una accion fisica	Consideramos que era preferible crear la clase tarjeta para evitar la sobrecarga a la persona responsable de los usos que le puede dar y permitimos a la tarjeta cambiar sus acciones o atributos sin llamar a la persona. Por ejemplo, si la tarjeta tiene "niveles" como premium y comun se maneja sin llamar a su dueño. Hicimos que la tarjeta conozca a su dueño ya que al momento de verificar si se puede usar requiere conocer la cantMenoresAcargo del mismo.			

Objetos	Sensor, heladera	patron observer (observador: Sensor, observado: sensorTemp). Un cambio en la temp o en un movimiento implica avisarle a la heladera y que esta verifique si debe inactivarse o no en base a ella.	1. Ponerle como atributo la temperatura a la heladera 2. Ponerle como atributo de la heladera al sensor de temp	1. La primera opcion no es logica ya que una heladera no conoce su temperatura, eso es trabajo del sensor. 2. La segunda opcion pierde flexibilidad ya que estamos fijando al sensor como parte de la heladera.			
Objetos	Tarjeta, Colaborador	Apertura registra la tarjeta y no al colaborador. La tarjeta está asociada a un colaborador y este puede tener varias tarjetas	Registrar al colaborador en lugar de la tarjeta	Flexibilidad: Registrar la tarjeta permite manejar situaciones en las que un colaborador tiene múltiples tarjetas o solicita una nueva. Evita confusiones sobre el colaborador y permite gestionar las tarjetas de manera independiente, como en casos de pérdida o actualización de la misma.			
Otro	Validador de contras, Condicion	tiene una lista del tipo Condicion que puede ser heredada por distintas clases que son validadas con un for en base a la password recibida	1. hacer un if por cada condicion	Flexibilidad: a futuro ya que, en caso de tener muchas codiciones no habria que agregar muchos "if" sino que se meten en la lista y cada una maneja su forma de validacion.			
Persistencia	SolicitudApertura, Heladera	El estado de la solicitud de apertura no se maneja; simplemente se realiza una consulta para verificar si existe una solicitud en las últimas 3 horas	Guardar el estado de cada solicitud de apertura y verificarlo cada vez que alguien intente abrir la heladera	Simplicidad: Usar una consulta reduce la complejidad de mantener estados innecesarios, lo que simplifica la implementación y mejora la eficiencia del sistema. Esto también evita la <u>sobrecarga de almacenamiento de estados</u> .			
Casos de Uso	Iniciar Sesion	No ponerla en el diagrama, sino que es una precondition del los casos de uso	poner el caso de uso "iniciar sesion" y que cada caso de uso incluya al mismo	Esto evitará la repetición innecesaria del caso de uso "Iniciar sesión" en múltiples lugares del diagrama y hará que el diagrama sea más claro y menos redundante.			
Casos de Uso	Colaboraciones	"Notificar al colaborador" es una postcondicion de cada colaboracion realizada	Ponerlo en el diagrama como caso de uso y que todas las colaboraciones lo incluyan	Esto evitará la repetición innecesaria de un include por cada colaboracion hacia el caso de uso "Notificar colaborador" y hará que el diagrama sea más claro y menos redundante.			
Casos de Uso	Colaboraciones	"Notificar al colaborador" es una postcondicion de cada colaboracion realizada	Ponerlo en el diagrama como caso de uso y que todas las colaboraciones lo incluyan	Esto evitará la repetición innecesaria de un include por cada colaboracion hacia el caso de uso "Notificar colaborador" y hará que el diagrama sea más claro y menos redundante.			
Arquitectura	Senores de movimiento y temperatura	Uso de Sensores para Movimiento y Temperatura (SensorMov y SensorTemp)	Los sensores son fundamentales para monitorear las condiciones físicas de las heladeras (por ejemplo, movimiento no autorizado o variaciones de temperatura). Cada sensor cuenta con un software propio (SW SensorMov y SW SensorTemp) que encapsula su lógica y envía datos al sistema central. Comparación:	Alternativa: Integrar todas las funciones de los sensores directamente en el software de la heladera física. Por qué no: Esto acoplaría demasiado las responsabilidades en un solo componente, dificultando el mantenimiento y las actualizaciones.			

Arquitectura	Senores de movimiento y temperatura	Uso de un Broker (RabbitMQ)	Comunicación directa entre los sensores y la base de datos o el servidor.	La alternativa crearía un fuerte acoplamiento y sería más difícil manejar problemas como pérdida de mensajes o fallos en el sistema receptor.			
Arquitectura	Recomendacion de puntos de colocacion de heladaras	Utilizar un servidor mock permite simular el comportamiento de sistemas externos (como sensores o proveedores de mapas) durante las pruebas, sin necesidad de depender de datos reales. Reduce el tiempo de desarrollo al permitir pruebas tempranas.	Realizar todas las pruebas directamente con los sistemas reales.	la alternativa podría retrasar el desarrollo debido a la dependencia de servicios que quizás no estén disponibles o configurados durante las pruebas iniciales.			
Arquitectura	vista de heladeras en mapas	Utilizar Google Maps permite obtener información precisa de geolocalización y ubicaciones recomendadas. Su API es ampliamente documentada y ofrece funcionalidades robustas, como calcular rutas y mostrar mapas en el navegador.	Usar APIs de otros proveedores, como OpenStreetMap o Mapbox.	Aunque son opciones válidas, Google Maps suele ser la elección preferida debido a su precisión y soporte técnico superior.			
Arquitectura	gestion de usuarios y seguridad	Auth0 facilita la autenticación y autorización segura, manejando aspectos como login, roles de usuario y tokens. Reduce la complejidad del desarrollo al externalizar la gestión de seguridad.	Implementar un sistema de autenticación propio.	implementar un sistema propio aumentaría la carga de desarrollo y requeriría un mantenimiento constante para abordar problemas de seguridad emergentes.			
Arquitectura	datos a persistir	MySQL es una base de datos robusta y ampliamente utilizada, ideal para almacenar información estructurada como registros de temperaturas, movimientos y usuarios. Ofrece un equilibrio entre rendimiento y facilidad de uso.	Usar una base de datos NoSQL, como MongoDB.	Aunque NoSQL es útil para datos no estructurados, las relaciones entre entidades en este sistema se gestionan mejor con un modelo relacional.			
Arquitectura	permitir a personas juridicas dar Reconocimientos extras a los colaboradores	Este servidor maneja funciones adicionales, como validaciones o procesamiento externo, reduciendo la carga en el servidor principal (Web Server). Permite dividir responsabilidades y mejorar la escalabilidad.	Incluir estas funciones en el servidor principal.	Incluirlo en el back podría impactar el rendimiento general del sistema al añadir más responsabilidades a un solo servidor.			

Arquitectura	Diseño de una API Back-End Desacoplada con un Front-End Independiente	Diseñar un back-end que expone una API RESTful permite que cualquier front-end (página web, aplicación móvil, etc.) consuma los mismos servicios. Esto es especialmente útil en un contexto donde gran parte de las interacciones son presenciales (retirar viandas, dejar donaciones, distribuir alimentos) pero pueden ser complementadas con aplicaciones digitales que faciliten la interacción. La posibilidad de incluir una aplicación	Diseñar el sistema con un enfoque monolítico donde la lógica del back-end esté acoplada al front-end actual.	el segundo enfoque dificultaría la integración de nuevas plataformas (como una app móvil) y aumentaría la complejidad de las actualizaciones, ya que cada nueva funcionalidad implicaría modificar tanto el back-end como el front-end. El desacople entre front-end y back-end permite mayor flexibilidad y escalabilidad, facilitando la incorporación de futuros clientes sin comprometer la arquitectura existente.			